Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 2

з дисципліни «Життєвий цикл розробки програмного забезпечення»

Тема: «Проектування архітектури та деталізоване проектування»

Виконали:
студентки групи ІП-44
Бідюк Марія
Хмелюк Діана

Перевірив:
Альбрехт Йосип
Омелянович

Київ 2026

ЗМІСТ

1. ВСТУП ТА МЕТА ПРОЄКТУВАННЯ..... 3

2. ВИБІР ТА ОБҐРУНТУВАННЯ АРХІТЕКТУРНОГО ПІДХОДУ..... 3

3. ВИСОКОРІВНЕВА АРХІТЕКТУРА СИСТЕМИ (COMPONENT DIAGRAM)..... 4

4. ДЕТАЛІЗОВАНЕ ПРОЄКТУВАННЯ КОМПОНЕНТІВ (UML-ДІАГРАМИ)..... 5

4.1. Статична структура: Діаграма класів (Class Diagram)..... 7

4.2. Динамічна поведінка: Діаграми послідовностей (Sequence Diagrams)..... 9

4.3. Життєвий цикл сутності: Діаграма станів (State Machine Diagram) 17

5. ОПИС ВЗАЄМОДІЇ КОМПОНЕНТІВ ТА ЗАХИСТ НЕФУНКЦІОНАЛЬНИХ ВИМОГ..... 18

6. ВИСНОВКИ..... 19

1. ВСТУП ТА МЕТА ПРОЄКТУВАННЯ

Метою даної лабораторної роботи є перетворення вимог, сформованих у специфікації SRS (Лабораторна робота №1), у повноцінне архітектурне рішення програмного забезпечення. Проєкт розробляється як веб-застосунок для знайомств, ключовими особливостями якого є **розподіл даних анкети на Відкриту та Закриту інформацію** (із доступом за взаємною згодою) та **автоматичний розрахунок сумісності за нумерологічною матрицею долі**.

Для забезпечення надійності, продуктивності та безпеки системи розроблено високорівневу макроархітектуру та деталізовані UML-моделі статичної й динамічної поведінки компонентів.

2. ВИБІР ТА ОБҐРУНТУВАННЯ АРХІТЕКТУРНОГО ПІДХОДУ

На основі специфікації вимог для реалізації системи обрано **архітектуру модульного моноліту (Monolithic Architecture)** на базі фреймворку **Spring Boot.**

Обґрунтування вибору:

1. **Забезпечення продуктивності:** відсутність мережевих затримок між окремими мікросервісами дозволяє виконувати розрахунки та обробку запитів у межах **до 2 секунд** для завантаження сторінок та **до 1 секунди** для обчислення матриці долі.

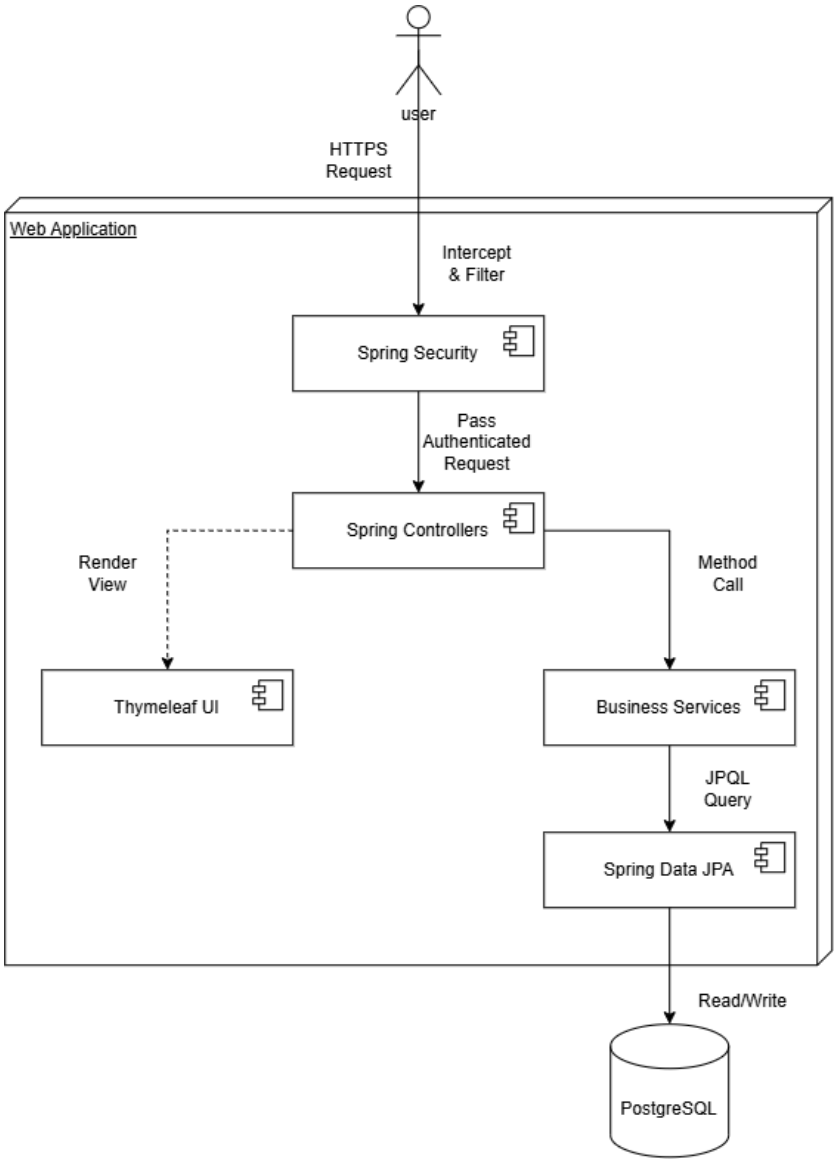
2. **Єдиний контекст безпеки:** використання **Spring Security** у межах монолітного застосунку дозволяє прозоро перехоплювати вхідні запити (SecurityFilterChain), керувати сесіями користувачів та гарантувати, що чутливі закриті дані фізично не потрапляють на клієнтську сторону до підтвердження запрошення.

3. **Простота розгортання та підтримки:** на початковому етапі розвитку продукту (MVP) монолітна структура суттєво спрощує

налаштування бази даних **PostgreSQL** та знижує накладні витрати на інфраструктурний моніторинг.

3. ВИСОКОРІВНЕВА АРХІТЕКТУРА СИСТЕМИ (COMPONENT DIAGRAM)

Високорівнева макроархітектура зображує розподіл системи на шари та схему проходження вхідного HTTPS-запиту від користувача.



Опис компонентів та шарів:

- **Security Layer (Spring Security):** перехоплює HTTPS-запити від користувача (Intercept & Filter), здійснює перевірку автентифікації та передає перевірений запит на рівень контролерів.

- **Presentation Layer (Spring Controllers & Thymeleaf UI):** отримує валідовані запити, викликає методи бізнес-сервісів та рендерить HTML-шаблони для користувача.
- **Business Layer (Business Services):** реалізує бізнес-логіку перевірки прав доступу, обміну запрошеннями та розрахунку матриці долі.
- **Persistence Layer (Spring Data JPA):** здійснює формування JPQL-запитів та обмін даними із реляційною СУБД PostgreSQL.

4. ДЕТАЛІЗОВАНЕ ПРОЄКТУВАННЯ КОМПОНЕНТІВ (UML-ДІАГРАМИ)

Відповідно до вимог методичних вказівок та стандартів проєктування складних програмних систем (ISO/IEC/IEEE 42010), деталізація високорівневої архітектури вимагає опису системи у трьох фундаментальних вимірах: **статичному (структура даних), динамічному міжшаровому (поведінка компонентів у часі)** та **динамічному об'єктному (життєвий цикл сутності)**.

Для даного проєкту обрано комплект із трьох UML-діаграм – **Діаграму класів (Class Diagram), Діаграму послідовностей (Sequence Diagram) та Діаграму станів (State Machine Diagram)** – з наступних архітектурних міркувань:

1. Діаграма класів (Class Diagram) – обґрунтування статичного аспекту:

- **критичність для проєкту:** ключовою особливістю системи є розмежування атрибутів анкети на відкриті (openData) та закриті (closedData), а також складні зв'язки між користувачами, анкетними інтересами та результатами нумерологічних обчислень;
- **чому це необхідно:** саме діаграма класів дозволяє зафіксувати контракти даних, типи атрибутів, модифікатори доступу (private/public) та JPA-відносини (@OneToOne, @ManyToMany, @ManyToOne). Вона є

прямим містком між вимогами приватності SRS та практичною побудовою реляційної бази даних PostgreSQL і Java-сутностей.

2. Діаграма послідовностей (Sequence Diagram) – обґрунтування динамічного аспекту:

- **критичність для проєкту:** головний бізнес-сценарій системи (UC-1: обмін запрошеннями та розрахунок сумісності) є скрізним і складається з трьох частин – надсилання, підтвердження та відхилення/скасування запрошення. Для кожної побудовано окрему діаграму послідовностей, що залучає акторів, Thymeleaf UI, InvitationController, бізнес-сервіси (InvitationService, DestinyMatrixService), сутність Invitation, JPA-репозиторії та СУБД PostgreSQL;
- **чому це необхідно:** діаграма послідовностей чітко документує часовий порядок виклику методів, міжшаровий обмін DTO-об'єктами та атомарність транзакцій. Вона демонструє, як саме Spring Security та бізнес-логіка блокують доступ до закритих даних до моменту настання події підтвердження запрошення.

3. Діаграма станів (State Machine Diagram) – обґрунтування динамічного об'єктного аспекту:

- **критичність для проєкту:** сутність запрошення (Invitation) є центральним елементом механізму розмежування приватності, яка змінює свій стан у відповідь на дії обох користувачів (надсилання, прийняття, відхилення або скасування);
- **чому це необхідно:** діаграма станів наочно документує життєвий цикл об'єкта Invitation через Enum InvitationStatus (PENDING → ACCEPTED / REJECTED / CANCELLED), фіксуючи бізнес-правила приватності та тригери переходів для всіх можливих гілок бізнес-логіки.

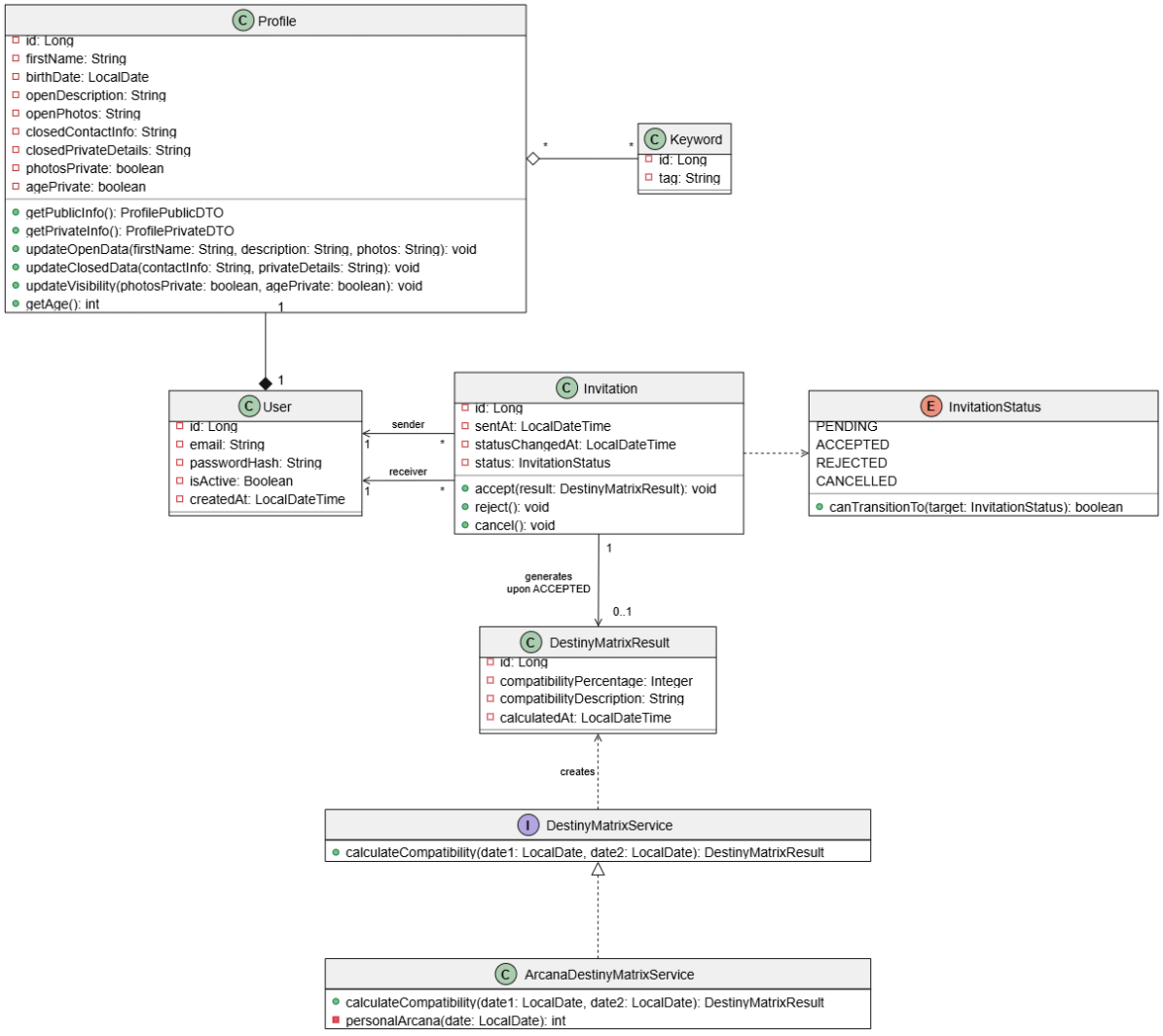
4. Синергія та повнота покриття комплекту з 3-х діаграм:

Поєднання Class Diagram + Sequence Diagram + State Machine Diagram надає необхідну інформацію для розробників та тестувальників:

- Class Diagram визначає, що і як зберігати;
- Sequence Diagram – хто, коли і в якому порядку обробляє запити;
- State Machine Diagram – як поводиться головна сутність на кожному етапі свого життя.

4.1. Статична структура: Діаграма класів (Class Diagram)

Діаграма класів відображає об'єктно-орієнтовану модель системи, її атрибути, типи даних, методи та зв'язки між сутностями.



Ключові елементи моделі:

1. **User (сутність облікового запису):** містить ідентифікатор id, email, passwordHash, прапорець isActive та дату створення createdAt.

Сутність зберігає лише дані облікового запису: реєстрацію виконує сервіс UserService, а вхід у систему – Spring Security.

2. **Profile (сутність анкети):** містить ім'я firstName і дату народження birthDate (використовується для розрахунку матриці долі, іншим користувачам показується лише вік – метод getAge()), а також фізично розділяє дані на відкриті (openDescription, openPhotos) та закриті (closedContactInfo, closedPrivateDetails). Метод getPublicInfo() повертає лише відкриту частину (ProfilePublicDTO), getPrivateInfo() – закриту (ProfilePrivateDTO); право доступу до закритої частини перевіряє бізнес-сервіс до виклику цього методу. Поля photosPrivate і agePrivate дозволяють власнику перенести фото та вік у закриту частину анкети (метод updateVisibility()).

3. **Keyword (теги/інтереси):** довідник інтересів користувача, пов'язаний із Profile відношенням «багато-до-багатьох».

4. **Invitation (запрошення):** зберігає дату відправлення sentAt, дату зміни стану statusChangedAt та поточний стан із перелічення InvitationStatus (PENDING, ACCEPTED, REJECTED, CANCELLED). Методи accept(result), reject() і cancel() змінюють стан лише з PENDING, що перевіряє метод InvitationStatus.canTransitionTo().

5. **DestinyMatrixService (інтерфейс сервісу обчислення сумісності):** оголошує метод calculateCompatibility(date1: LocalDate, date2: LocalDate): DestinyMatrixResult. Його реалізує клас ArcanaDestinyMatrixService, який обчислює нумерологічну матрицю долі. Завдяки інтерфейсу алгоритм розрахунку можна замінити без змін у бізнес-логіці запрошень (принципи Open/Closed та Dependency Inversion).

6. **DestinyMatrixResult (сутність результату сумісності):** збережена сутність даних (POJO), яка містить відсоток сумісності compatibilityPercentage, опис compatibilityDescription та дату розрахунку calculatedAt.

Типи зв'язків та їхнє обґрунтування:

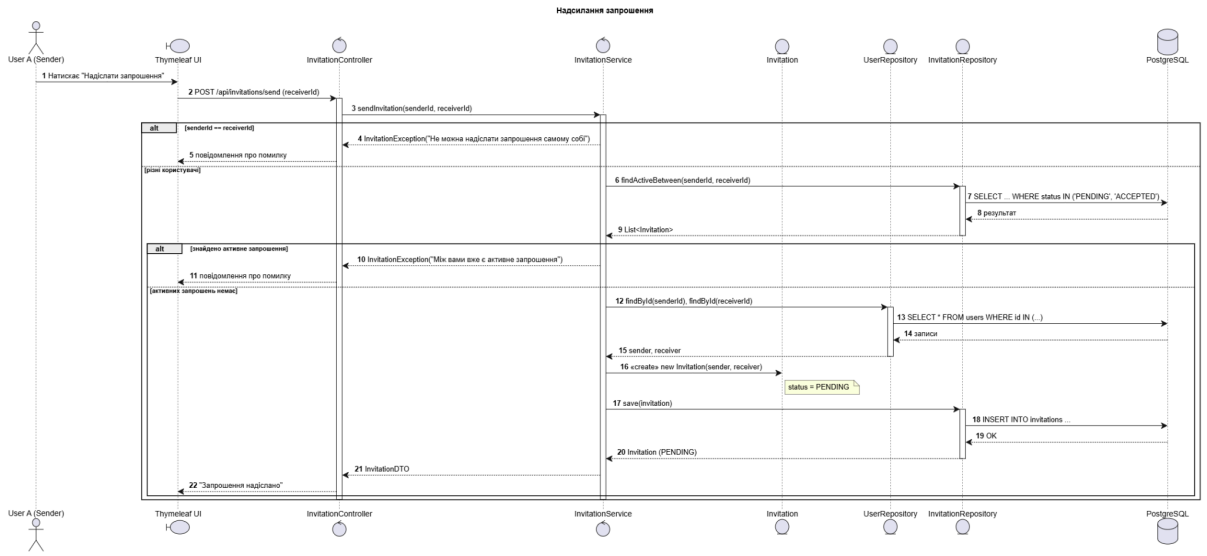
- **User – Profile (композиція):** заповнений чорний ромб вказує, що анкета належить єдиному користувачу і не існує без нього.
- **Profile – Keyword (агрегація):** порожній ромб вказує, що теги інтересів є окремим довідником і зберігаються у системі навіть при видаленні анкети.
- **User – Invitation (асоціація):** дві спрямовані асоціації з кратністю 1 до *, що розділяють ролі відправника та отримувача запрошення.
- **Invitation – DestinyMatrixResult (асоціація):** кратність 1 до 0...1 показує, що розрахунок матриці долі створюється лише при переході запрошення у статус ACCEPTED.
- **Invitation – InvitationStatus (залежність):** пунктирна стрілка показує використання класів-перелічень станів.
- **DestinyMatrixService – DestinyMatrixResult (залежність):** пунктирна стрілка з підписом creates від класу DestinyMatrixService напряму до DestinyMatrixResult, яка показує, що сервіс формує та обчислює об'єкт результату.
- **ArcanaDestinyMatrixService – DestinyMatrixService (реалізація):** пунктирна стрілка з порожнім трикутником показує, що клас реалізує інтерфейс сервісу розрахунку сумісності.

4.2. Динамічна поведінка: Діаграми послідовностей (Sequence Diagrams)

Діаграми послідовностей відображають виконання в часі головного бізнес-сценарію UC-1: «Обмін запрошеннями, розкриття закритих даних та розрахунок сумісності за матрицею долі». Щоб кожну діаграму було легко читати, сценарій розділено на три частини: надсилання запрошення, підтвердження запрошення з розрахунком сумісності, а також відхилення

та скасування запрошення. На кожній діаграмі показано основний перебіг і альтернативні шляхи (фрагменти alt), а смуги активації вказують, який компонент ініціює кожен виклик.

4.2.1. Надсилання запрошення



1. Користувач A у формі Thymeleaf UI натискає кнопку «Надіслати запрошення».

2. На InvitationController надходить запит POST /api/invitations/send з ідентифікатором отримувача (receiverId).

3. Контролер викликає метод sendInvitation(senderId, receiverId) у бізнес-сервісі InvitationService.

Альтернатива: користувач надсилає запрошення самому собі.

4. InvitationService виявляє, що відправник і отримувач збігаються, і повертає контролеру InvitationException.

5. InvitationController показує на UI повідомлення про помилку, запрошення не створюється.

Основний перебіг: відправник і отримувач різні.

6. InvitationService звертається до InvitationRepository через метод findActiveBetween(senderId, receiverId).

7. InvitationRepository виконує SQL-запит SELECT у PostgreSQL і шукає між двома користувачами запрошення зі статусом PENDING або ACCEPTED.

8. PostgreSQL повертає результат вибірки.

9. InvitationRepository передає InvitationService список знайдених запрошень.

Альтернатива: між користувачами вже існує активне запрошення.

10. InvitationService повертає контролеру InvitationException «Між вами вже є активне запрошення».

11. InvitationController показує на UI повідомлення про помилку, повторне запрошення не створюється.

Основний перебіг: активних запрошень немає.

12. InvitationService запитує в UserRepository відправника та отримувача за їхніми ідентифікаторами.

13. UserRepository виконує SQL-запит SELECT до таблиці users.

14. PostgreSQL повертає записи користувачів.

15. UserRepository повертає InvitationService об'єкти sender і receiver.

16. InvitationService створює новий об'єкт Invitation зі статусом PENDING.

17. InvitationService викликає метод save(invitation) у InvitationRepository.

18. InvitationRepository виконує запит INSERT у PostgreSQL.

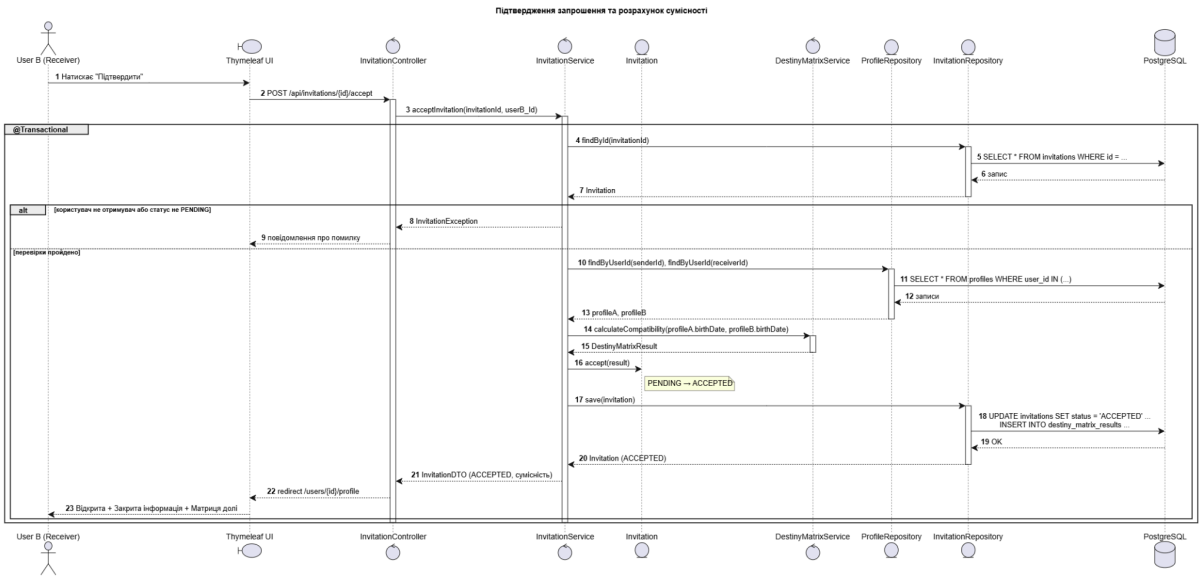
19. PostgreSQL підтверджує успішне збереження.

20. InvitationRepository повертає збережене запрошення зі статусом PENDING.

21. InvitationService повертає контролеру InvitationDTO.

22. InvitationController показує на UI повідомлення «Запрошення надіслано».

4.2.2. Підтвердження запрошення та розрахунок сумісності



Усі дії InvitationService у цьому сценарії виконуються в межах однієї транзакції (@Transactional, рамка на діаграмі). Якщо будь-який крок завершиться помилкою, жодна зміна не буде збережена в базі даних.

1. Користувач Б у своєму інтерфейсі Thymeleaf UI натискає «Підтвердити».

2. На InvitationController надходить запит POST /api/invitations/{id}/accept.

3. Контролер викликає метод acceptInvitation(invitationId, userB_Id) у InvitationService.

4. InvitationService робить запит findById(invitationId) до InvitationRepository.

5. InvitationRepository виконує SQL-запит SELECT до таблиці invitations.

6. PostgreSQL повертає запис запрошення.

7. InvitationRepository повертає InvitationService об'єкт Invitation.

Альтернатива: підтверджує не отримувач, або запрошення вже оброблене.

8. Якщо поточний користувач не є отримувачем запрошення або запрошення вже не перебуває в статусі PENDING, InvitationService повертає контролеру InvitationException.

9. InvitationController показує на UI повідомлення про помилку, стан запрошення не змінюється.

Основний перебіг: перевірки пройдено.

10. InvitationService запитує в ProfileRepository анкети відправника та отримувача.

11. ProfileRepository виконує SQL-запит SELECT до таблиці profiles.

12. PostgreSQL повертає записи анкет.

13. ProfileRepository повертає анкети profileA і profileB.

14. InvitationService викликає метод calculateCompatibility(profileA.birthDate, profileB.birthDate) у DestinyMatrixService і передає дати народження обох користувачів.

15. DestinyMatrixService обчислює сумісність і повертає об'єкт DestinyMatrixResult.

16. InvitationService викликає метод accept(result) об'єкта Invitation: стан запрошення змінюється з PENDING на ACCEPTED, до нього прив'язується результат розрахунку.

17. InvitationService викликає метод save(invitation) у InvitationRepository.

18. InvitationRepository виконує запит UPDATE до таблиці invitations і INSERT результату сумісності до таблиці destiny_matrix_results.

19. PostgreSQL підтверджує збереження.

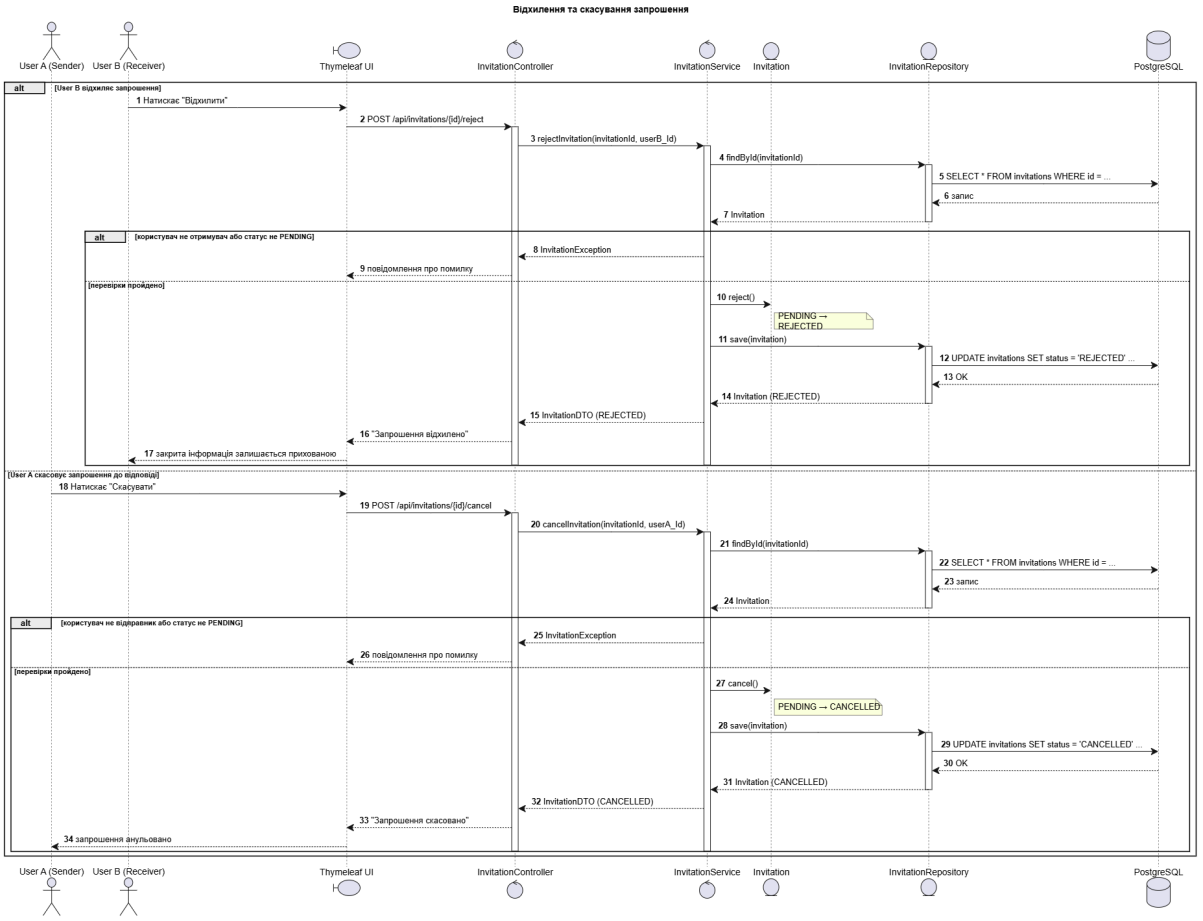
20. InvitationRepository повертає оновлене запрошення зі статусом ACCEPTED.

21. InvitationService повертає контролеру InvitationDTO зі статусом ACCEPTED і показниками сумісності.

22. InvitationController перенаправляє користувача на сторінку анкети (redirect /users/{id}/profile).

23. Thymeleaf UI відображає користувачу Б відкриту та закриту інформацію анкети і показники матриці долі.

4.2.3. Відхилення та скасування запрошення



Діаграма охоплює два альтернативні способи завершити запрошення без підтвердження. Обидва змінюють стан запрошення, але не відкривають закриту інформацію і не запускають розрахунок сумісності.

Альтернатива: користувач Б відхиляє запрошення.

1. Користувач Б у Thymeleaf UI натискає «Відхилити».

2. На InvitationController надходить запит POST /api/invitations/{id}/reject.

3. Контролер викликає метод rejectInvitation(invitationId, userB_Id) у InvitationService.

4. InvitationService робить запит findById(invitationId) до InvitationRepository.

5. InvitationRepository виконує SQL-запит SELECT до таблиці invitations.

6. PostgreSQL повертає запис запрошення.

7. InvitationRepository повертає InvitationService об'єкт Invitation.

8. Якщо поточний користувач не є отримувачем або запрошення вже не перебуває в статусі PENDING, InvitationService повертає контролеру InvitationException.

9. InvitationController показує на UI повідомлення про помилку, стан запрошення не змінюється.

10. Якщо перевірки пройдено, InvitationService викликає метод reject() об'єкта Invitation: стан змінюється з PENDING на REJECTED.

11. InvitationService викликає метод save(invitation) у InvitationRepository.

12. InvitationRepository виконує запит UPDATE до таблиці invitations.

13. PostgreSQL підтверджує збереження.

14. InvitationRepository повертає оновлене запрошення зі статусом REJECTED.

15. InvitationService повертає контролеру InvitationDTO зі статусом REJECTED.

16. InvitationController показує на UI повідомлення «Запрошення відхилено».

17. Закрита інформація обох користувачів залишається прихованою.

Альтернатива: користувач А скасовує запрошення до відповіді.

18. Користувач А в Thymeleaf UI натискає «Скасувати».

19. На InvitationController надходить запит POST /api/invitations/{id}/cancel.

20. Контролер викликає метод cancelInvitation(invitationId, userA_Id) у InvitationService.

21. InvitationService робить запит findById(invitationId) до InvitationRepository.

22. InvitationRepository виконує SQL-запит SELECT до таблиці invitations.

23. PostgreSQL повертає запис запрошення.

24. InvitationRepository повертає InvitationService об'єкт Invitation.

25. Якщо поточний користувач не є відправником або запрошення вже не перебуває в статусі PENDING, InvitationService повертає контролеру InvitationException.

26. InvitationController показує на UI повідомлення про помилку, стан запрошення не змінюється.

27. Якщо перевірки пройдено, InvitationService викликає метод cancel() об'єкта Invitation: стан змінюється з PENDING на CANCELLED.

28. InvitationService викликає метод save(invitation) у InvitationRepository.

29. InvitationRepository виконує запит UPDATE до таблиці invitations.

30. PostgreSQL підтверджує збереження.

31. InvitationRepository повертає оновлене запрошення зі статусом CANCELLED.

32. InvitationService повертає контролеру InvitationDTO зі статусом CANCELLED.

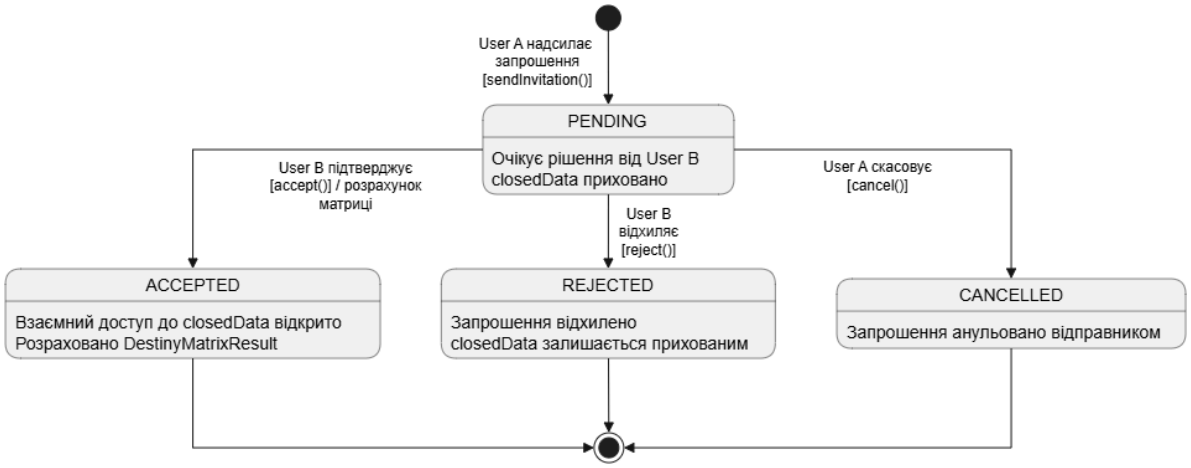
33. InvitationController показує на UI повідомлення «Запрошення скасовано».

34. Запрошення анульовано відправником, закрита інформація залишається прихованою.

Разом три діаграми покривають усі переходи життєвого циклу запрошення, описані на діаграмі станів (розділ 4.3): PENDING → ACCEPTED, PENDING → REJECTED і PENDING → CANCELLED. Вони також показують, що зміну стану й збереження запрошення завжди ініціює InvitationService.

4.3. Життєвий цикл сутності: Діаграма станів (State Machine Diagram)

Для деталізації правил зміни станів ключового об'єкта предметної області – запрошення (Invitation) – побудовано діаграму станів. Вона описує переходи між значеннями перелічення InvitationStatus під впливом дій користувачів та бізнес-правил системи приватності.



Опис станів та переходів:

1. **Початковий стан → PENDING (очікування):**

- **тригер:** користувач А надсилає запит через метод sendInvitation();
- **особливість:** запрошення отримує стан PENDING. На цьому етапі закриті дані обох користувачів (closedContactInfo, closedPrivateDetails) залишаються повністю прихованими.

2. **Перехід PENDING → ACCEPTED (прийнято):**
   - **тригер:** користувач Б натискає «Підтвердити» (метод accept());
   - **побічний ефект:** автоматично запускається DestinyMatrixService.calculateCompatibility(), зберігається DestinyMatrixResult, і система відкриває взаємний доступ до закритих даних анкет.
3. **Перехід PENDING → REJECTED (відхилено):**
   - **тригер:** користувач Б натискає «Відхилити» (метод reject());
   - **особливість:** доступ до закритих даних блокується, розрахунок сумісності не виконується.
4. **Перехід PENDING → CANCELLED (скасовано):**
   - **тригер:** користувач А скасовує власне запрошення до отримання відповіді (метод cancel());
   - **особливість:** запрошення переходить у термінальний стан анулювання.

5. ОПИС ВЗАЄМОДІЇ КОМПОНЕНТІВ ТА ЗАХИСТ НЕФУНКЦІОНАЛЬНИХ ВИМОГ

Розроблена архітектура гарантує виконання всіх нефункціональних вимог специфікації SRS:

1. **Безпека та приватність:**

- закрита інформація (closedContactInfo, closedPrivateDetails) фізично відокремлена від відкритих DTO і передається з бекенду лише після перевірки статусу запрошення ACCEPTED у InvitationService;
- паролі користувачів хешуються алгоритмом BCrypt перед збереженням у User;
- з'єднання захищене HTTPS, а дії доступу до анкет і входи в систему записуються в журнал аудиту.

2. **Продуктивність та швидкодія:**

- розрахунок матриці долі винесено в окремий легковаговий сервіс DestinyMatrixService, що гарантує час обчислення до 1 секунди;
- використання JPA-індексів у PostgreSQL за ключовими словами забезпечує завантаження пошукових вибірок до 2 секунд.

3. **Надійність:**

- транзакційність у InvitationService (@Transactional) гарантує, що зміна статусу запрошення та створення запису матриці долі виконуються атомарно;
- повідомлення про помилки формуються українською мовою на окремій сторінці без технічних подробиць; стан системи доступний через ендпоінт моніторингу.

6. ВИСНОВКИ

У ході виконання Лабораторної роботи №2 було розроблено повноцінний документ «Архітектура програмного забезпечення» та деталізовані UML-моделі для веб-застосунку «Сайт знайомств з розрахунком сумісності за матрицею долі».

Основні результати роботи:

1. **Обґрунтування макроархітектури:** обрано та обґрунтовано підхід модульного моноліту на базі Spring Boot, Thymeleaf та PostgreSQL. Дане рішення є оптимальним для розробки командою з двох осіб, оскільки забезпечує чітку ізоляцію доменних модулів, зводить до мінімуму конфлікти у версіонуванні коду (Git), гарантує високу швидкодію (завантаження сторінок до 2 секунд, розрахунок сумісності до 1 секунди) та простоту розгортання єдиного артефакту.

2. **Високорівневе проєктування (Component Diagram):** побудовано діаграму компонентів, яка демонструє багатошарову структуру системи та трасування вхідного HTTPS-запиту крізь периметр безпеки (Spring Security), рівень представлення (Controllers / Thymeleaf UI), сервіси бізнес-логіки (Services) та шар абстракції даних (Spring Data JPA / PostgreSQL).

3. **Деталізація статичної структури (Class Diagram v2.0):** створено оновлену модель класів, яка реалізує ключову вимогу приватності (розділення анкетних даних на openData та closedData). Для дотримання принципу єдиної відповідальності (Single Responsibility) обчислювальний функціонал винесено в окремий інтерфейс DestinyMatrixService з реалізацією ArcanaDestinyMatrixService, яка формує сутність DestinyMatrixResult та зафіксовано коректні UML-зв'язки (композиція, агрегація, спрямовані асоціації та залежності).

4. **Деталізація міжшарової динаміки (Sequence Diagram):** побудовано три діаграми послідовностей для головного бізнес-сценарію UC-1 (надсилання, підтвердження з розрахунком сумісності, відхилення та скасування запрошення) з альтернативними шляхами обробки помилок, які відображають часовий порядок виклику методів, обмін DTO-об'єктами та транзакційну взаємодію компонентів системи.

5. **Деталізація життєвого циклу сутності (State Machine Diagram):** додано діаграму станів для центрального об'єкта приватності – Invitation. Вона описує всі можливі переходи між значеннями Enum InvitationStatus (PENDING → ACCEPTED / REJECTED / CANCELLED), фіксуючи умови взаємного розкриття закритих даних та розгалуження бізнес-логіки.

Побудований комплекс UML-моделей гарантує дотримання нефункціональних вимог безпеки, надійності та продуктивності і слугує

готовим технічним завданням для скрам-планування беклогу, розробки кодової бази, інтеграції компонентів та підготовки тест-кейсів.